

#Task 2: Dataset Exploration

```
import pandas as pd
from google.colab import files
uploaded = files.upload()
labels = pd.read_csv('labels.csv')

print(labels.head())
print(labels['class'].value_counts())
```

Choose Files labels.csv

labels.csv(text/csv) - 17056 bytes, last modified: 5/17/2026 - 100% done
Saving labels.csv to labels (1).csv

	filename	class
0	images/normal/normal_001.png	normal
1	images/normal/normal_002.png	normal
2	images/normal/normal_003.png	normal
3	images/normal/normal_004.png	normal
4	images/normal/normal_005.png	normal

class

normal	120
scratch	120
dent	120
stain	120

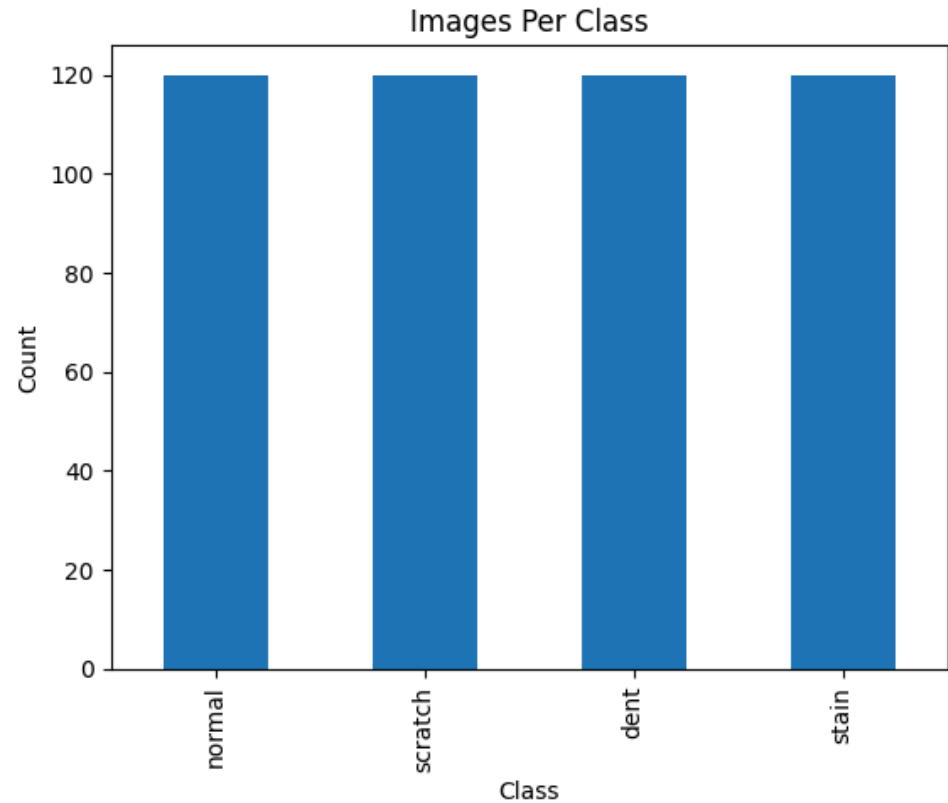
Name: count, dtype: int64

```
classes = labels['class'].unique()
print(classes)
print("Number of classes:", len(classes))
```

['normal' 'scratch' 'dent' 'stain']
Number of classes: 4

```
import matplotlib.pyplot as plt

labels['class'].value_counts().plot(kind='bar')
plt.title('Images Per Class')
plt.xlabel('Class')
plt.ylabel('Count')
plt.show()
```



```
from google.colab import files
from PIL import Image
import matplotlib.pyplot as plt

uploaded = files.upload()
```

dent_001.png
dent_001.png(image/png) - 10949 bytes, last modified: 5/17/2026 - 100% done
Saving dent_001.png to dent_001 (2).png

```
from google.colab import files
from PIL import Image
import matplotlib.pyplot as plt
```

```
uploaded = files.upload()
```

normal_001.png
normal_001.png(image/png) - 9503 bytes, last modified: 5/17/2026 - 100% done
Saving normal_001.png to normal_001 (1).png

```
from google.colab import files
from PIL import Image
import matplotlib.pyplot as plt
```

```
uploaded = files.upload()
```

scratch_001.png
scratch_001.png(image/png) - 10833 bytes, last modified: 5/17/2026 - 100% done
Saving scratch_001.png to scratch_001 (1).png

```
from google.colab import files
from PIL import Image
import matplotlib.pyplot as plt
```

```
uploaded = files.upload()
```

stain_001.png
stain_001.png(image/png) - 11150 bytes, last modified: 5/17/2026 - 100% done
Saving stain_001.png to stain_001 (1).png

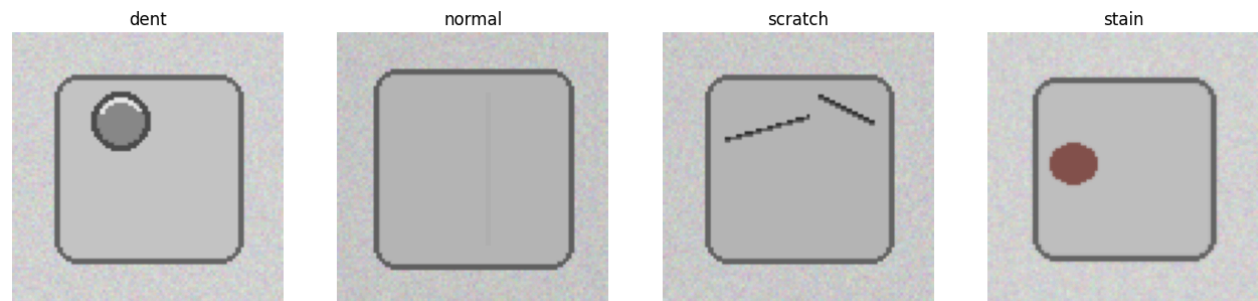
```
from PIL import Image
import matplotlib.pyplot as plt

# Image file names
image_names = [
    "dent_001 (2).png",
    "normal_001 (1).png",
    "scratch_001 (1).png",
    "stain_001 (1).png"
]

# Display images
fig, axes = plt.subplots(1, 4, figsize=(16,4))

for i, img_name in enumerate(image_names):
    img = Image.open(img_name)
    axes[i].imshow(img)
    axes[i].axis('off')
    axes[i].set_title(img_name.split('_')[0])

plt.show()
```



```
print(labels['class'].value_counts())
```

```
class
normal      120
scratch     120
dent        120
stain       120
Name: count, dtype: int64
```

#Task 3: Image Preprocessing

```
# Import libraries
import os
import numpy as np
import pandas as pd
from PIL import Image
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

import tensorflow as tf
from tensorflow.keras.preprocessing.image import img_to_array
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Dataset Information
image_names = [
    "dent_001 (2).png",
    "normal_001 (1).png",
    "scratch_001 (1).png",
    "stain_001 (1).png"]

labels = [
    "dent",
    "normal",
    "scratch",
    "stain"]

# Display Sample Images
fig, axes = plt.subplots(1, 4, figsize=(16,4))

for i, img_name in enumerate(image_names):

    img = Image.open(img_name)

    axes[i].imshow(img)
    axes[i].axis('off')
    axes[i].set_title(labels[i])
plt.show()

# Image Dimensions
for img_name in image_names:
    img = Image.open(img_name)
    print(f"{img_name} -> {img.size}")

# Image Preprocessing
IMG_SIZE = 128

X = []
y = []

for img_name, label in zip(image_names, labels):

    img = Image.open(img_name)

    # Resize image
    img = img.resize((IMG_SIZE, IMG_SIZE))

    # Convert to array
    img_array = img_to_array(img)

    # Normalize pixel values
    img_array = img_array / 255.0

    X.append(img_array)
    y.append(label)

# Convert to numpy arrays
X = np.array(X)
y = np.array(y)
print("Image dataset shape:", X.shape)

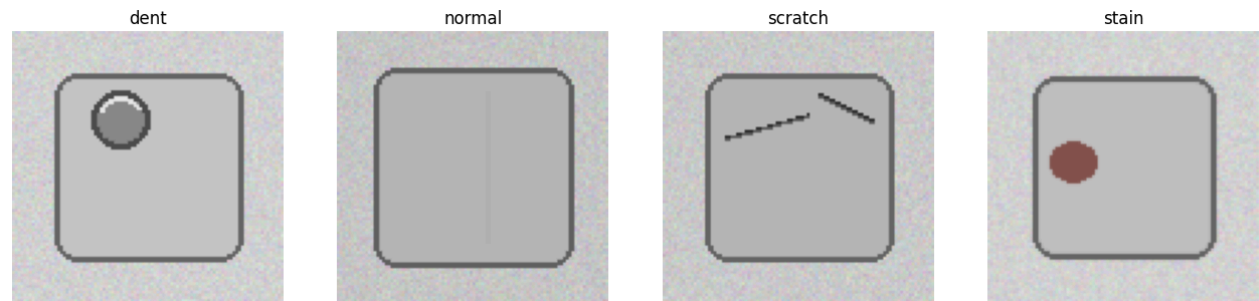
# Encode Labels
encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)
print("Encoded Labels:", y_encoded)
```

```
# Train Test Split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.25,
    random_state=42
)
print("Training images:", len(X_train))
print("Testing images:", len(X_test))

# Data Augmentation
train_datagen = ImageDataGenerator(
    rotation_range=20,
    zoom_range=0.2,
    horizontal_flip=True
)

train_datagen.fit(X_train)
```



```
dent_001 (2).png -> (96, 96)
normal_001 (1).png -> (96, 96)
scratch_001 (1).png -> (96, 96)
stain_001 (1).png -> (96, 96)
Image dataset shape: (4, 128, 128, 3)
Encoded Labels: [0 1 2 3]
Training images: 3
Testing images: 1
```

#Task 4: CNN Model Creation

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Conv2D,
    MaxPooling2D,
    Flatten,
    Dense,
    Dropout
)

# Number of classes
num_classes = len(set(y_encoded))

# CNN Model
model = Sequential([

    # First Convolution Layer
    Conv2D(
        32,
        (3,3),
        activation='relu',
        input_shape=(128,128,3)
    ),

    # Pooling Layer
    MaxPooling2D(pool_size=(2,2)),

    # Second Convolution Layer
    Conv2D(
        64,
        (3,3),
        activation='relu'
    ),

    # Pooling Layer
    MaxPooling2D(pool_size=(2,2)),
```

```
# Third Convolution Layer
Conv2D(
    128,
    (3,3),
    activation='relu'
),

# Pooling Layer
MaxPooling2D(pool_size=(2,2)),

# Flatten Layer
Flatten(),

# Dense Layer
Dense(128, activation='relu'),

# Dropout
Dropout(0.5),

# Output Layer
Dense(num_classes, activation='softmax')
])

# Model Summary
model.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning:
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 4)	516

Total params: 3,305,156 (12.61 MB)
Trainable params: 3,305,156 (12.61 MB)
Non trainable params: 0 (0.00 MB)

#Task 5: Model Training and Evaluation

```
history = model.fit(
    train_datagen.flow(X_train, y_train, batch_size=2),
    validation_data=(X_test, y_test),
    epochs=15
)
```

Epoch 1/15
2/2  3s 368ms/step - accuracy: 0.0000e+00 - loss: 2.6302 - val_accuracy: 0.0000e+00
Epoch 2/15
2/2  0s 185ms/step - accuracy: 0.3333 - loss: 1.7153 - val_accuracy: 0.0000e+00
Epoch 3/15
2/2  0s 133ms/step - accuracy: 0.3333 - loss: 1.3641 - val_accuracy: 0.0000e+00
Epoch 4/15
2/2  0s 122ms/step - accuracy: 0.3333 - loss: 1.2402 - val_accuracy: 0.0000e+00
Epoch 5/15
2/2  0s 125ms/step - accuracy: 0.0000e+00 - loss: 1.2910 - val_accuracy: 0.0000e+00
Epoch 6/15
2/2  0s 176ms/step - accuracy: 0.0000e+00 - loss: 1.2356 - val_accuracy: 0.0000e+00
Epoch 7/15
2/2  0s 160ms/step - accuracy: 0.6667 - loss: 1.0875 - val_accuracy: 0.0000e+00
Epoch 8/15
2/2  0s 126ms/step - accuracy: 0.0000e+00 - loss: 1.2810 - val_accuracy: 0.0000e+00
Epoch 9/15
2/2  0s 128ms/step - accuracy: 0.3333 - loss: 1.0989 - val_accuracy: 0.0000e+00
Epoch 10/15

```
2/2 ----- 1s 274ms/step - accuracy: 0.3333 - loss: 1.1194 - val_accuracy: 0.0000e+00
Epoch 11/15
2/2 ----- 1s 531ms/step - accuracy: 0.3333 - loss: 0.9517 - val_accuracy: 0.0000e+00
Epoch 12/15
2/2 ----- 1s 971ms/step - accuracy: 0.3333 - loss: 1.0394 - val_accuracy: 0.0000e+00
Epoch 13/15
2/2 ----- 2s 290ms/step - accuracy: 0.0000e+00 - loss: 1.6603 - val_accuracy: 0.0000e+00
Epoch 14/15
2/2 ----- 0s 127ms/step - accuracy: 0.0000e+00 - loss: 1.2124 - val_accuracy: 0.0000e+00
Epoch 15/15
2/2 ----- 0s 160ms/step - accuracy: 0.3333 - loss: 1.2972 - val_accuracy: 0.0000e+00
```

```
plt.figure(figsize=(12,5))

# Accuracy Plot
plt.subplot(1,2,1)

plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')

plt.title('Training vs Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')

plt.legend()

# Loss Plot
plt.subplot(1,2,2)

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

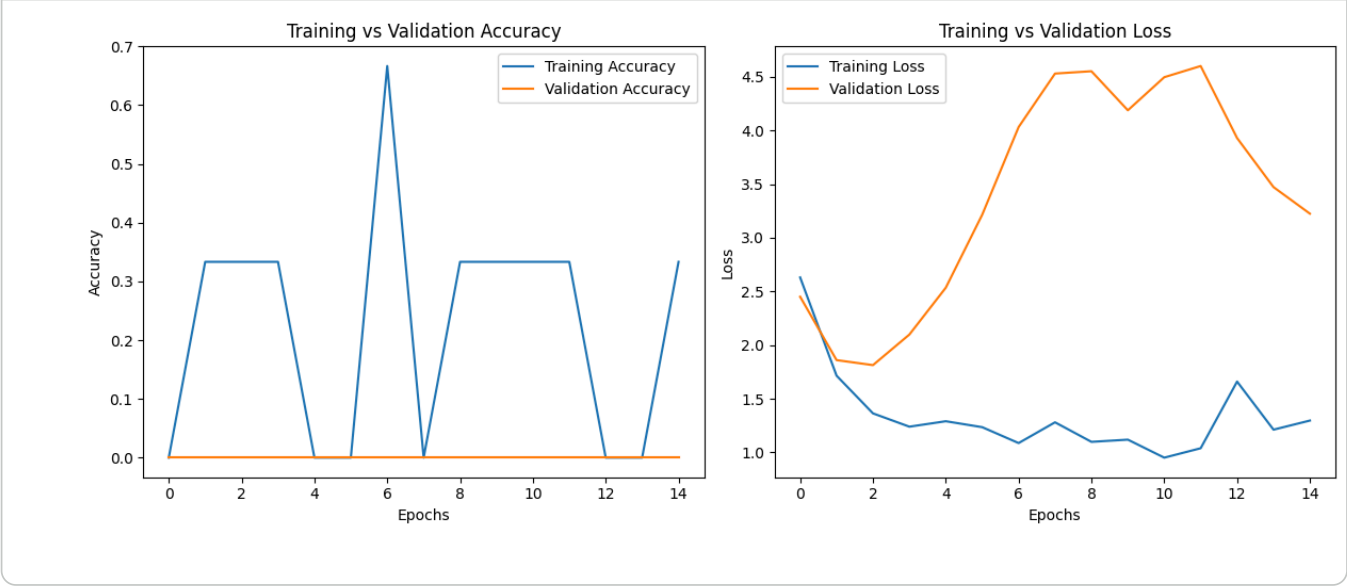
plt.title('Training vs Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')

plt.legend()

plt.tight_layout()

plt.savefig('accuracy_loss_curves.png')

plt.show()
```



```
test_loss, test_accuracy = model.evaluate(X_test, y_test)

print("Test Accuracy:", test_accuracy)
print("Test Loss:", test_loss)

1/1 ----- 0s 53ms/step - accuracy: 0.0000e+00 - loss: 3.2241
Test Accuracy: 0.0
Test Loss: 3.2240724563598633
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

# Predictions
predictions = model.predict(X_test)
```

```
y_pred = np.argmax(predictions, axis=1)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

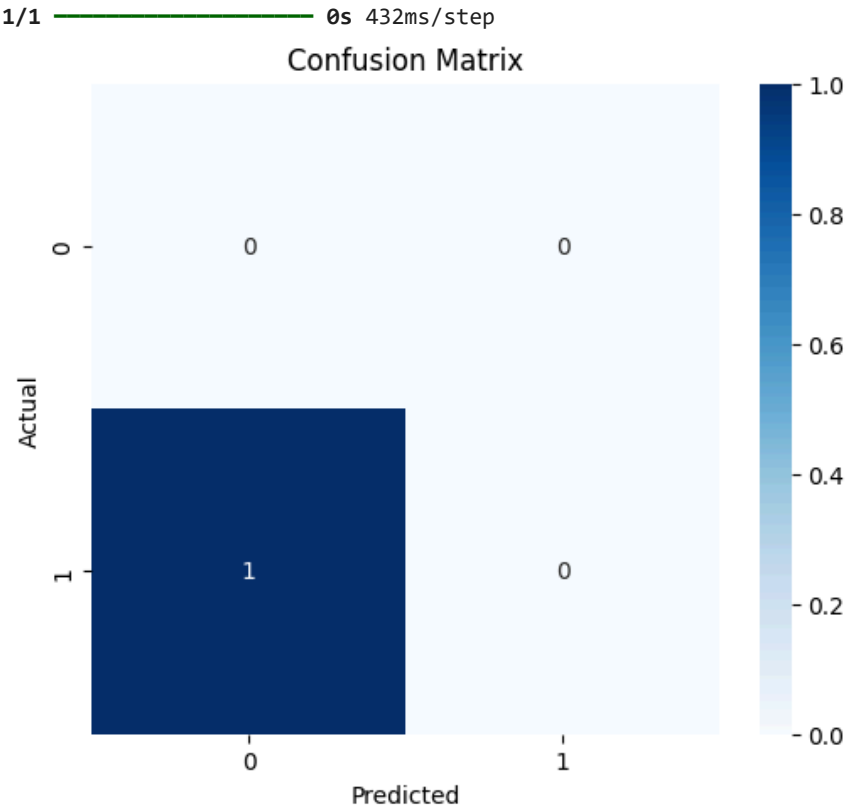
# Plot
plt.figure(figsize=(6,5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.savefig("confusion_matrix.png")

plt.show()
```



```
# Display predictions

fig, axes = plt.subplots(1, len(X_test), figsize=(12,4))

if len(X_test) == 1:
    axes = [axes]

for i, ax in enumerate(axes):

    ax.imshow(X_test[i])

    predicted_label = encoder.inverse_transform([y_pred[i]])[0]
    actual_label = encoder.inverse_transform([y_test[i]])[0]

    ax.set_title(
        f"Pred: {predicted_label}\nActual: {actual_label}"
    )

    ax.axis('off')

plt.tight_layout()

plt.savefig("prediction_outputs.png")

plt.show()
```

Pred: dent
Actual: normal

